

ChatGPT



[Build AI and ML applications with .NET and C#](#)

Show moreShow less

That link is Microsoft's "**Build AI and ML applications with .NET and C#**" resource. It's a good starting point if you're a C#/.NET developer who wants to move into AI rather than switching entirely to Python.

Microsoft's current .NET AI material covers:

- **Generative AI & LLMs** — chat, summarization, content generation.
- **AI agents** — using Microsoft Agent Framework to build workflows and agents.
- **RAG** — embeddings, vector search, document ingestion, and chat-with-your-data applications.
- **AI APIs** — OpenAI, Azure OpenAI, Azure AI Foundry, Gemini, Ollama, Amazon Bedrock, and others through .NET abstractions.
- **Traditional ML** — ML.NET for building and training machine-learning models directly with C#.
- **Computer vision, speech, classification, and automation.**  [Microsoft Learn+1](#)

[Microsoft's .NET AI & ML page](#)

If you're learning this from scratch

I'd follow this order:

1. C#/.NET fundamentals → 2. ASP.NET Core/Web APIs → 3. AI/LLM fundamentals → 4. Microsoft.Extensions.AI → 5. RAG → 6. AI agents → 7. ML.NET

You **don't need to become a data scientist first**. For modern AI application development, the valuable skill is often knowing how to integrate models into real applications—APIs, databases, authentication, RAG, tools/functions, agents, and deployment.

Microsoft specifically recommends learning paths covering chat applications, summarization, chatting with your data, function/tool calling, image generation, and ML.NET.  [Microsoft Learn](#)

If your goal is **getting job-ready in AI + C#/.NET**, I can also turn this Microsoft material into a **step-by-step 3-6 month roadmap with projects and what to learn in each week**.



yes

Absolutely. If your goal is to become **job-ready in AI/ML application development with C#/.NET**, I'd focus less on academic ML at first and more on building production-style AI applications.

6-Month .NET + AI Roadmap

Month 1 — Strong C#/.NET foundation

Goal: Be comfortable building backend applications before adding AI.

Learn:

- C# fundamentals and modern C#
- LINQ, async/await, generics
- Dependency injection
- Interfaces and clean architecture
- .NET configuration and logging
- REST APIs
- ASP.NET Core
- Entity Framework Core
- SQL basics
- Git/GitHub
- Unit testing with xUnit

Project:

Build a **Task Management REST API** with:

- ASP.NET Core Web API
- SQL database
- Entity Framework Core
- Authentication
- CRUD operations
- Unit tests
- Swagger/OpenAPI

Don't move on until you can build a reasonably complete API without following a tutorial line-by-line.

Month 2 — AI & LLM fundamentals

Goal: Understand what you're actually programming when you call an AI model.

Learn:

- What LLMs are
- Tokens and context windows
- System/user/assistant messages
- Temperature and generation

- Structured outputs
- Embeddings
- Function/tool calling
- Streaming responses
- Prompt engineering
- Model limitations
- Hallucinations
- AI security basics
- API costs and latency

Then learn the .NET AI abstractions, particularly **Microsoft.Extensions.AI**.

Project:

Turn your task API into an **AI Task Assistant**.

For example:

"I have three hours this afternoon. Which tasks should I prioritize?"

The AI should retrieve the user's tasks and produce a structured recommendation.

Month 3 — RAG

This is one of the most valuable areas to learn for AI application development.

RAG = Retrieval-Augmented Generation.

Learn:

1. Document ingestion
2. Text extraction
3. Chunking
4. Embeddings
5. Vector databases/search
6. Similarity search
7. Retrieval
8. Prompt construction
9. Citations
10. Evaluation

Project:

Build a **Chat With Your Documents** application.

Architecture:

PDF / Word / Web documents

↓
Text extraction

↓
Chunking

↓
Embeddings

↓
Vector store

↓
User question

↓
Similarity search

↓
Relevant chunks

↓
LLM

↓
Answer + sources

Build both the backend API and a simple web UI.

This becomes your first serious portfolio project.

Month 4 — AI agents & tools

Now move beyond simple chatbots.

Learn:

- Function calling
- Tool calling
- Agents
- Agent memory
- Multi-step workflows
- Human approval
- Agent orchestration
- Microsoft Agent Framework
- Reliability and guardrails

For example, create an agent that can:

User

↓

AI Agent

— Search documents

— Query database

— Create task

— Send notification

— Ask for human approval

Project:

Build an **AI Business Assistant**.

Give it tools such as:

- Search company documents
- Query customer information
- Create support tickets
- Summarize tickets
- Generate reports

The important part isn't making the agent "clever."

It's demonstrating that you can safely connect an LLM to **real software functionality**.

Month 5 — Production AI engineering

This is where you differentiate yourself from someone who has simply followed an AI tutorial.

Learn:

- Docker
- Azure fundamentals
- CI/CD
- Secrets management
- Authentication/authorization
- Logging
- Monitoring
- Rate limiting
- Caching
- Retry policies
- Streaming
- Token/cost tracking
- Prompt/version management
- AI evaluation
- Security
- Responsible AI

Learn to answer questions like:

What happens if the AI API goes down?

How do we prevent users from accessing another customer's documents?

How much does each request cost?

How do we detect hallucinations?

How do we evaluate whether a new prompt is actually better?

Project:

Take your RAG application and deploy it as a real cloud application.

For example:

```

Frontend
↓
ASP.NET Core API
↓
AI service
↓
Vector search
↓
SQL / Blob Storage

```

Containerize it with Docker and deploy it to Azure.

Month 6 — ML + portfolio + interviews

Only now would I put significant time into traditional machine learning.

Learn:

- Supervised vs unsupervised learning
- Regression
- Classification
- Clustering
- Training/validation/test sets
- Overfitting
- Feature engineering
- Model evaluation
- Precision/recall/F1
- Basic neural-network concepts
- ML.NET

Project: Build a traditional ML application with ML.NET.

Example:

Customer churn prediction

```

Customer data
↓
Feature engineering
↓
ML.NET model
↓
Prediction
↓
ASP.NET Core API
↓
Web dashboard

```

This gives you exposure to both:

Your 3 Portfolio Projects

I'd prioritize these three rather than building ten tiny chatbot demos.

1. AI Document Assistant

Stack:

- C#
- ASP.NET Core
- Microsoft.Extensions.AI
- LLM API
- Embeddings
- Vector database/search
- SQL
- Docker

Features:

- Upload documents
- Ask questions
- RAG
- Source citations
- Conversation history
- Authentication

2. AI Business Agent

Build an agent that can actually **do things**.

Example:

"Find customers who haven't been contacted in 30 days, summarize their history, and create follow-up tasks."

The agent needs to interact with APIs/database tools.

This demonstrates a much more advanced skill than:

"Make a chatbot."

3. ML Prediction API

Use ML.NET to create a traditional machine-learning model.
 Expose it through ASP.NET Core and create a small dashboard.
 That demonstrates that you understand both sides of the ecosystem.

What I'd Learn Alongside Everything

Don't neglect these:

Skill	Importance
C#	★★★★★
ASP.NET Core	★★★★★
SQL	★★★★★
REST APIs	★★★★★
Git	★★★★★
LLM APIs	★★★★★
RAG	★★★★★
AI agents	★★★★★
Azure	★★★★★
Docker	★★★★★
Testing	★★★★★
ML fundamentals	★★★★
ML.NET	★★★★
Python	★★★★

Notice that **Python isn't number one**.

If you're specifically targeting **.NET AI application developer / AI engineer roles**, becoming excellent at C# and backend engineering is a major advantage.

You should nevertheless learn enough Python to read AI/ML code and work with the broader AI ecosystem.

A Weekly Schedule

If you can dedicate about **10-12 hours/week**:

Monday — 1.5h

Learn new concept

Tuesday — 1.5h

Code along / small exercises

Wednesday — 1.5h

AI/ML theory

Thursday — 1.5h

Build your project

Saturday — 4h

Deep project work

Sunday — 1-2h

Review + GitHub documentation

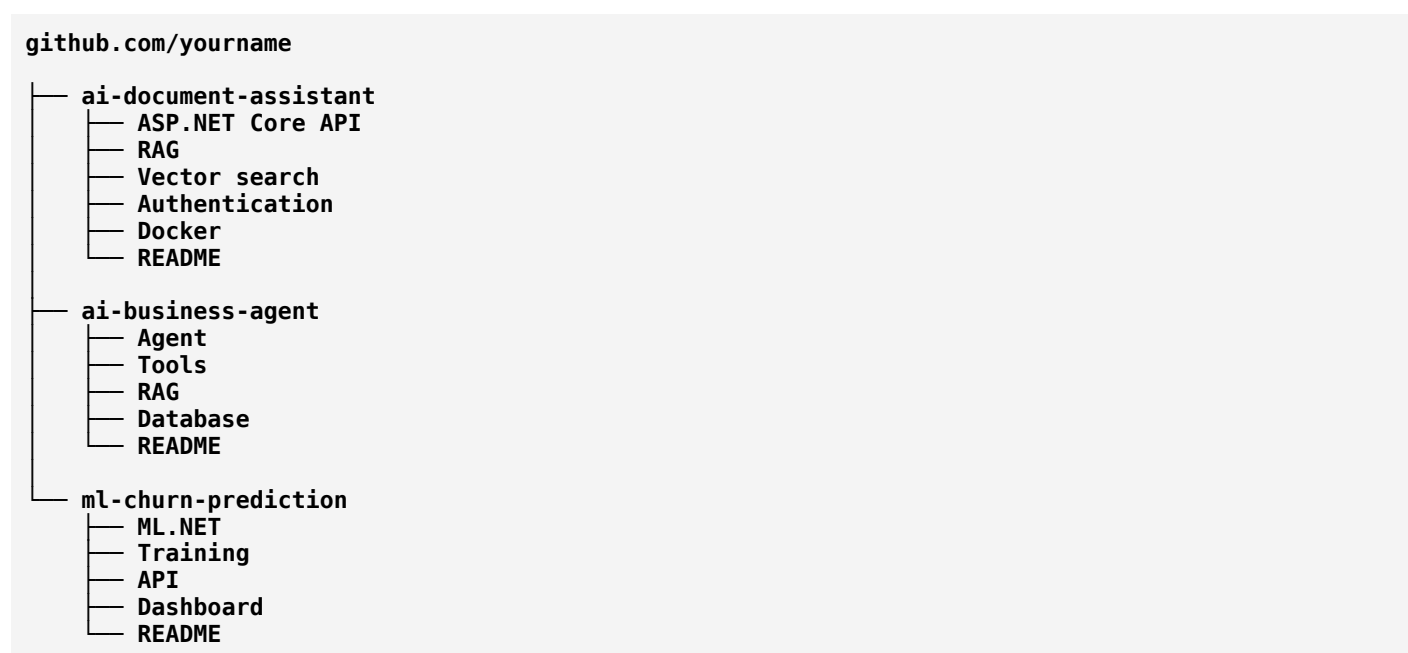
The important ratio is roughly:

30% learning → 70% building.

Don't spend six months watching courses.

What Your GitHub Should Look Like

By the end, I'd want to see something like:



Each README should explain:

- What problem it solves
- Architecture
- Technologies used
- How the AI works
- How to run it
- Screenshots
- Example requests/responses
- Design decisions
- Limitations
- Future improvements

That's much stronger in an interview than simply saying **"I completed an AI course."**

The end goal

After six months, you want to be able to confidently say:

"I build production-oriented AI applications in C#/.NET, including LLM integrations, RAG systems, AI agents, APIs, databases, and cloud deployments. I also understand traditional machine learning and can build ML.NET models."

That's a very practical target—and the Microsoft material you linked is a good foundation for getting there.

ChatGPT is AI and can make mistakes.